DURABLE. / ENGINEERING DOCUMENTATION

Durable Job Queue

Developer Handbook

Go + PostgreSQL + deterministic simulation

Documentation snapshot / 18 October 2026

Reliable work, even after a worker stops

This project records background tasks in PostgreSQL, lets Go workers process them, and recovers unfinished work through expiring leases. A browser dashboard makes job status, retries, results, and failure simulations visible.

It is a runnable learning and portfolio project. Its built-in handlers echo JSON and calculate SHA-256 hashes. Report generation and other business workflows are examples of applications that could use the queue; they are not included handlers.

> The core guarantee: processing attempts can repeat, while each successful job has one committed database result. Recovery requires surviving database data and active workers.

What is included

- A durable PostgreSQL queue and concurrent Go workers.
- A CLI, HTTP API, and embedded browser dashboard.
- A deterministic simulator with reproducible failure schedules.
- Contract, integration, worker, API, and simulation tests.
- Docker Compose and a GitHub Actions workflow.

Who this guide is for

Use it as an onboarding document for a new contributor, a reference while debugging, or preparation for explaining the project in an interview. Commands assume a terminal opened at the project root. Shell examples use POSIX syntax.

01 / Reading map

Start with the outcome you need

For a first run, begin on page 5. To understand recovery, read pages 7-11. API and GUI usage are on pages 12-14; tests are on pages 15-16. Contributors should continue to page 17, and portfolio preparation is on page 19.

Contents

Pg.	Topic	Pg.	Topic
3	Purpose and guarantees	12	API: create jobs
4	Architecture and code map	13	API: inspect and simulate
5	Docker quick start	14	Dashboard walkthrough
6	Local Go and configuration	15	Deterministic simulation
7	Data model and lifecycle	16	Tests and evidence
8	Enqueue and idempotency	17	Extending the project
9	Claims, leases, and fencing	18	Operations and troubleshooting
10	Workers and retries	19	GitHub, LinkedIn, and interviews
11	Transactional completion	20	Glossary and maintenance

How to read code examples

Shell commands are intended to be runnable from the repository root. SQL excerpts and pseudocode explain the protocol; the source files contain complete validation, transaction handling, and error paths. Example IDs such as `42` or `JOB_ID` are illustrative; actual job IDs are generated strings.

The source tree is the implementation authority. This guide describes the checked-in code at the snapshot date, rather than an independent specification of future versions. Recorded test outcomes describe previous local verification; they are not new test executions caused by reading or generating this PDF.

Useful companion documents

- `README.md`: entry point, commands, and API summary.
- `docs/ARCHITECTURE.md`: transaction and failure-boundary details.
- `docs/DEMO.md`: a short presentation and process-kill demonstration.
- `docs/VALIDATION.md`: verification scope and links to saved evidence.
- `docs/CV_AND_INTERVIEW.md`: defensible CV wording and technical answers.

02 / Purpose and guarantees

Why build this?

Imagine a user requests a report and closes the website. The request should still be remembered. If the process doing the work crashes, the system should be able to try again instead of silently forgetting the request. A background queue separates accepting work from completing it.

The project explores that reliability problem through a small, inspectable implementation. Go runs the application and workers. PostgreSQL stores task state and results. The browser interface helps demonstrate what happens when jobs succeed, fail, or need another attempt.

What the implementation promises

Property	Meaning in this project
Durable acceptance	After enqueue commits, the job exists in PostgreSQL. Persistence depends on PostgreSQL and its storage.
Safe producer retries	Repeating the same queue and key with the same kind and payload returns the original job.
Recoverable ownership	An expired running lease can be reclaimed if attempts remain.
Fenced completion	A worker with a superseded ownership version cannot commit completion.
Atomic result commit	One transaction inserts the result and marks the job succeeded.
Explicit terminal failure	A job that exhausts its attempts stays visible as `dead`.

What recovery means

The worker retries the task from the beginning. There is no saved instruction pointer, memory snapshot, or automatic checkpoint inside a handler. A running worker renews its lease; after a crash, renewals stop and the database deadline eventually passes. Another worker can then claim the job.

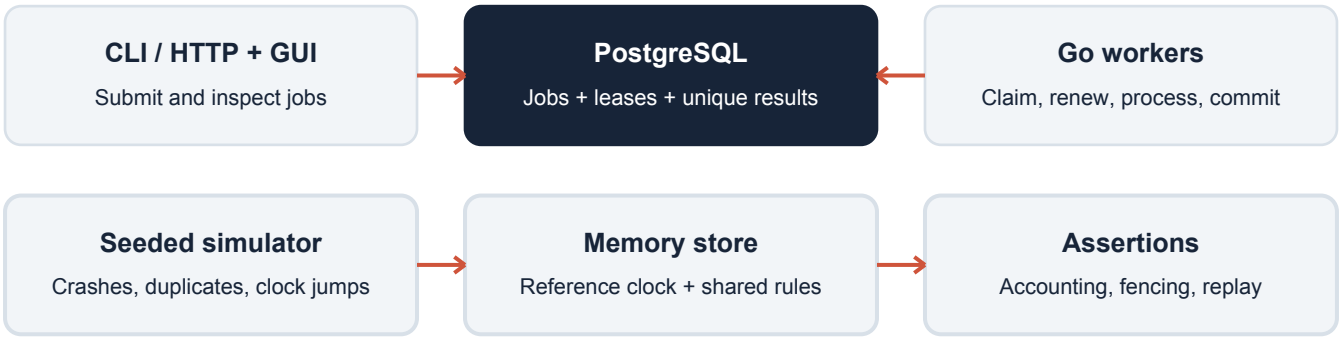
The software does not restart a crashed server or replace a failed machine. Someone or a process supervisor must start a worker again. The supplied Compose file does not configure an automatic restart policy.

> Exactly-once committed database results do not mean exactly-once execution. Two attempts can perform computation, especially after a lease expires. External emails, payments, and HTTP calls need their own deduplication design.

Present scope

This is a local development and portfolio application. It includes `echo` and `sha256` handlers, correctness tests, and a working GUI. It does not include production authentication, database failover, throughput measurements, report generation, or a deployed public service.

03 / Architecture and code map



Producers submit through the CLI or HTTP API. Both use the same queue protocol. Workers poll PostgreSQL, perform computation outside the claim transaction, and return a result through a completion transaction. The GUI calls the HTTP API. The simulator uses the memory reference store and never injects faults into live database jobs.

Location	Responsibility
`cmd/dq/main.go`	CLI flags, startup, server lifecycle, demo handlers
`queue/queue.go`	Job types, store interface, validation, leases, shared transitions
`queue/worker.go`	Polling, worker slots, heartbeats, cancellation, processing
`queue/postgres/`	Durable store, schema, inspection queries, integration tests
`queue/memory/`	Reference store with an injected clock
`simulation/`	Seeded fault scheduling, invariants, replay and fuzz tests
`internal/httpapi/`	Routes, request validation, embedded dashboard
`internal/queuetest/`	Behavioral contracts exercised against both stores
`scripts/`	CLI demo and real-process crash smoke test

Why use PostgreSQL?

The same database can hold durable queue state, a unique result, and application SQL changes inside one transaction. Row locks provide coordinated claims. This is useful for studying reliability without adding a separate message broker.

The trade-offs include polling, row contention, duplicate-enqueue writes, and long-term table growth. This design does not promise strict FIFO processing, unlimited scalability, or fairness under unlimited contention.

Why use Go?

Goroutines provide bounded parallel worker slots and heartbeat loops. Context cancellation coordinates shutdown and lease loss. The reliability property comes from persistent state, transaction rules, and ownership checks; it is not a special recovery feature of the language.

04 / Docker quick start

Requirements

Install Docker Desktop or Docker Engine with Compose and start its engine. The complete container workflow does not require local Go, Node.js, or Python. The first build needs access to container images and Go modules.

```
docker compose up --build -d
docker compose ps
curl http://127.0.0.1:8080/healthz
```

Open `http://127.0.0.1:8080` in a browser. A healthy API responds with `{"status":"ok"}`. Compose starts PostgreSQL, waits for readiness, runs `migrate`, and then starts the API and worker. The migration container exiting successfully is expected.

Submit your first job

```
curl -sS http://127.0.0.1:8080/v1/jobs \
  -H 'Content-Type: application/json' --data-binary @- <<'JSON'
{"kind":"echo","idempotency_key":"hello-001",
 "payload":{"message":"hello"}}
JSON
```

The response contains the job ID. Inspect the job in the dashboard or run `curl -sS http://127.0.0.1:8080/v1/jobs`. The worker should eventually mark it `succeeded`; its result contains the echoed payload. The POST itself means accepted, not finished.

Check logs and stop safely

```
docker compose logs --tail=100 worker api
docker compose down
```

`down` retains the named database volume. Starting the stack again reuses its data. Do not add `--volumes` or `-v` unless you intend to delete the development database.

Ports are local: API `127.0.0.1:8080`, PostgreSQL `127.0.0.1:55432`. The `queue` username/password in Compose are development values. Publishing this repository does not require exposing these local services.

05 / Local Go and configuration

Use Go 1.25 or newer when developing locally. PostgreSQL can still run in Docker. A separate Node.js build is unnecessary because the web assets are embedded in the Go executable.

```
docker compose up -d postgres
cp .env.example .env
set -a
. ./.env
set +a
make build
bin/dq migrate
bin/dq worker -concurrency 4
```

In a second terminal at the same project root, load the environment again and start the API:

```
set -a
. ./.env
set +a
bin/dq serve
```

The program reads environment variables; it does not automatically load `.env`. If the Compose API already occupies port 8080, stop that service first or pass `-addr 127.0.0.1:8081`. Stop the Compose worker when you want only your local worker processing the default queue.

Setting	Default or requirement
`DATABASE_URL`	Required by database commands; see `.env.example`
`TEST_DATABASE_URL`	Enables PostgreSQL tests; use a development database
Worker `-queue`	`default`; each worker consumes its configured queue
Worker `-concurrency`	4 slots; supported range 1-256
Worker `-lease`	15 seconds; heartbeat interval is one third of the lease
Enqueue `-attempts`	5; permitted range 1-100
Enqueue `-delay`	0; maximum 30 days
Serve `-addr`	`127.0.0.1:8080` for the local CLI

Useful CLI commands

```
bin/dq enqueue -key demo-001 -kind echo -payload '{"n":1}'
bin/dq list -queue default -limit 20
bin/dq get -id JOB_ID
bin/dq results
bin/dq simulate -seed 42 -steps 1000 -jobs 30 -workers 5
```

Replace `JOB_ID` with a returned ID. Run `bin/dq COMMAND -h` for flags. Rebuild with `make build` after changing Go code or embedded web assets.

06 / Data model and lifecycle

The schema lives in `queue/postgres/schema.sql`. Migration creates missing tables and indexes. It is an initial schema installer, not a versioned schema migration framework.

Table / fields	Meaning
`dq_jobs.id`, `queue`, `kind`	Identity, logical queue, and handler name
`idempotency_key`, `fingerprint`	Producer deduplication and payload-conflict detection
`payload`	JSONB input for the handler
`status`	`pending`, `running`, `succeeded`, or `dead`
`attempts`, `max_attempts`	Claimed attempts so far and their upper bound
`owner`, `lease_version`, `lease_until`	Current ownership and database-time deadline
`available_at`	Earliest time a pending job may be claimed
`created_at`, `updated_at`, `last_error`	Inspection and troubleshooting information
`dq_results.job_id`, `value`, `committed_at`	One JSONB result per successful job

State transitions

```
pending --claim-----------------------> running
running --successful transaction-----> succeeded
running --failure, attempts remain---> pending (delayed)
running --failure, budget exhausted--> dead
running --expired lease, budget left-> running (new version)
running --expired final attempt------> dead (on polling sweep)
```

Both `succeeded` and `dead` are terminal through the queue API. A dead job is retained failure information; it is not proof that the task completed successfully. There is no built-in endpoint that replays or deletes terminal jobs.

Database constraints and indexes

- Unique `(queue, idempotency_key)` prevents a duplicate accepted job in that queue.
- `dq_results.job_id` is a primary key referencing the job; a second result row for the same job cannot be inserted.
- Checks restrict statuses, attempt counts, and lease presence. Only a running job has a non-null lease deadline.
- Partial indexes cover pending jobs by availability and running jobs by lease expiry.

The job ID stays the same across attempts. The attempt counter and lease version rise on every claim, including a claim followed immediately by a crash. Jobs are retained rather than removed when processing begins.

07 / Enqueue and idempotency

The path is `POST /v1/jobs` or CLI `enqueue`, then `Request.Normalize`, then `Store.Enqueue`. The caller supplies an idempotency key representing one logical request.

The same request can be sent safely again

```
queue = default, key = report-001, payload = {"n":1}
First submission  -> creates job A
Same submission   -> returns job A
Same key, {"n":2} -> conflict
Different queue   -> a separate key namespace
```

If the producer loses the response after the insert commits, it does not know whether the operation succeeded. It should retry with the original key. Generating a new key would represent new work and could create another job.

How the code decides whether requests match

`Normalize` validates the request, parses its JSON, normalizes object order and insignificant whitespace, and hashes the handler kind plus normalized payload using SHA-256. The store compares this fingerprint when a key already exists.

Numeric spellings such as `1` and `1.0` remain distinct for fingerprinting. PostgreSQL stores JSONB, so handlers must not depend on original whitespace or object key order. PostgreSQL JSONB restrictions still apply to submitted values.

The first accepted request fixes the retry budget and initial schedule. Repeating the key does not change those settings, even if a later request supplies different `max_attempts` or `delay_ms`. Keys remain reserved while the retained job exists, including after success or dead-lettering.

Concurrent submissions

The database unique constraint decides the winning row. The SQL conflict path performs a no-op key update and returns that winning row, then checks its fingerprint. A different kind or payload yields `ErrConflict`, mapped to HTTP `409`.

This avoids a race in which a concurrent insert is detected but cannot be retrieved by a same-statement snapshot. The trade-off is an extra row version for duplicate submissions. High-volume use would require measuring this write overhead and planning retention and vacuum behavior.

Input boundaries

Queue and kind are nonblank and at most 128 bytes; keys are nonblank and at most 256 bytes. Payloads must be valid JSON and at most 1 MiB in the store interface. The HTTP limit applies to the whole request body, so its practical payload limit is smaller.

Source: `queue/queue.go` (`Normalize`) and `queue/postgres/postgres.go` (`Enqueue`).

08 / Claims, leases, and fencing

`Claim` opens a short transaction, sweeps up to 100 expired final-attempt jobs to `dead`, and selects one eligible job. This selection uses a row lock:

```
SELECT ... FROM dq_jobs
WHERE queue = $1 AND attempts < max_attempts AND (
  (status = 'pending' AND available_at <= clock_timestamp())
  OR
  (status = 'running' AND lease_until <= clock_timestamp())
)
ORDER BY available_at, id
LIMIT 1 FOR UPDATE SKIP LOCKED;
```

Locked rows are skipped so another worker can claim different work. After taking the lock, the code reads database time again and rechecks eligibility. It updates ownership, increments attempts and version, sets the deadline, and commits. The slow handler runs after this transaction ends.

A concrete crash timeline

Database time	What happens
10:00:00	Worker A claims job 42, version 1, deadline 10:00:15.
10:00:05	A renews the lease; the new deadline is 10:00:20.
10:00:07	A stops responding after a crash or pause. Heartbeats stop.
10:00:21	Worker B claims the expired job, version 2, if attempts remain.
10:00:22	If A was paused and resumes, its version-1 completion is rejected.

Why the version matters

A lease deadline tells the queue when reassignment is permitted. A fencing version tells it which assignment is current. Ownership validation checks job ID, owner, version, running status, and an unexpired deadline. The database row lock serializes completion against a new claim.

An old handler can still be computing after reassignment. Fencing prevents that old attempt from committing through the queue; it cannot cancel arbitrary external effects that already happened.

Which clock is trusted?

PostgreSQL `clock_timestamp()` is authoritative. A forward clock jump can expire leases early; a backward jump can delay recovery. An expired but unreassigned lease can appear live again after a backward jump. A superseded version stays invalid. Eventual progress assumes database time eventually advances, workers are active, and failures stop long enough to finish or exhaust the budget.

Source: `Claim`, `Extend`, and `locked` in the PostgreSQL store; `Acquire` and `CheckLease` in `queue/queue.go`.

09 / Workers and retry policy

`Worker.Run` starts a bounded number of goroutines. Each slot repeatedly calls `Claim`, waits when no job is eligible, and processes its claimed job. The default poll interval is 200 milliseconds. Each slot has a distinct owner suffix.

```
repeat until cancelled:
    claim one eligible job
    if no job or a claim error: wait, then try again
    start lease renewal loop
    invoke the handler
    stop renewal loop
    if cancelled or lease renewal failed: leave for recovery
    if handler failed: schedule retry or mark dead
    otherwise: attempt transactional completion
```

Heartbeats and cancellation

The renewal loop ticks every `LeaseDuration / 3`. With the default 15-second lease, it renews every 5 seconds. Renewal itself has a timeout of one third of the lease. A renewal failure cancels the handler context, and the worker does not acknowledge cancelled work.

Handlers must honor their context. Go cannot forcibly terminate an arbitrary goroutine. A handler that ignores cancellation can keep a slot occupied or delay shutdown. Database ownership checks still reject its superseded completion token.

The `invoke` wrapper turns a handler panic into an ordinary attempt failure. A successful handler must return valid JSON. A failed completion call is logged and left for recovery; the worker cannot assume whether an ambiguous database response means commit or rollback.

Bounded retries

Attempts count claims, not just error returns. The default budget is five. An explicit handler failure sets the job back to `pending` with a delayed `available_at`, unless the budget is exhausted, in which case it becomes `dead`.

```
delay = min(2^(attempt - 1) seconds, 60 seconds) + jitter
jitter = deterministic fraction of the base delay, below 25%
```

The base delays are approximately 1, 2, 4, 8, 16, 32, and 60 seconds. Crashed leases are eligible as soon as they expire; they do not receive this additional failure backoff.

Demonstrate retries

```
bin/dq enqueue -key retry-001 \
  -payload '{"message":"hello","demo_fail_until":2}'
```

With an active worker and the default budget, the first two attempts fail and the third can succeed. Use a fresh key to repeat the demonstration. Source: `queue/worker.go`, `Backoff`, and `Reject`.

10 / Transactional completion

`Complete` delegates to PostgreSQL `CompleteTx` without a business-write callback. Completion locks the job and reads database time after acquiring that lock. It validates the grant, inserts one result, marks the job `succeeded`, and commits everything together.

```
BEGIN
  lock the job row
  if already succeeded with this winning token: return success
  validate owner, version, running status, and deadline
  execute optional application SQL using this transaction
  insert the unique result
  mark the job succeeded
COMMIT
```

A replay of the winning token returns success without replacing the original result or running its callback again. A stale token is rejected. The replay path is a successful no-op.

Business writes in the same transaction

The PostgreSQL-specific `CompleteTx` method accepts a callback using the supplied `pgx.Tx`. An application can update its own table there. If that callback or commit fails before a successful transaction commit, its database effects, result, and acknowledgement are not partially committed.

The standard `queue.Handler` returns only JSON and an error; the bundled worker calls `Complete`. To use application writes, integrate `CompleteTx` into an explicit processing/completion path. Do not commit inside the handler and then let the ordinary worker acknowledge separately.

The callback must use the provided transaction, remain short and context-aware, and never commit, roll back, or call an external service. The row lock remains held during it. A deadline passing during the callback does not let a different worker reclaim that locked row before the transaction ends.

Failure boundaries

Failure point	Expected recovery
Enqueue response lost	Resubmit the original idempotency key.
Crash after claim or during computation	Wait for lease expiry; another attempt may start.
Failure before completion commits	Transaction rolls back; the lease can recover.
Commit succeeds but response is lost	Replay of the winning token is a no-op; the saved success remains.
Old worker returns after reassignment	Its lower version is rejected.
Crash on the final allowed attempt	A later claim sweep retains the job as `dead`.

External side effects need destination-side idempotency or a separately designed outbox/inbox protocol. Those mechanisms are not part of the current implementation. Source: `CompleteTx` and its PostgreSQL integration tests.

11 / HTTP API: create jobs

Base URL for the local stack: `http://127.0.0.1:8080`. The API returns JSON. `POST /v1/jobs` accepts one object and rejects unknown fields or extra JSON values.

```
{
  "queue": "default",
  "kind": "echo",
  "idempotency_key": "api-demo-001",
  "payload": {"message": "hello"},
  "max_attempts": 5,
  "delay_ms": 0
}
```

Field	Meaning and validation
`queue`	Optional; defaults to `default`; at most 128 bytes.
`kind`	Required nonblank handler name; at most 128 bytes.
`idempotency_key`	Required nonblank logical request key; at most 256 bytes.
`payload`	Required valid JSON; whole HTTP body is limited to 1 MiB.
`max_attempts`	Omitted or zero becomes 5; otherwise 1-100.
`delay_ms`	Initial delay in milliseconds; 0-2,592,000,000.

Response semantics

- `202 Accepted`: returns the job, including ID, status, attempts, owner, lease version, and timestamps. A `Location` header points to `/v1/jobs/{id}`. An identical duplicate submission also returns 202 and the existing job, even if it has already finished.
- `400 Bad Request`: malformed or oversized input, invalid limits, unknown fields, or additional JSON values.
- `409 Conflict`: the key already belongs to a different kind or payload in the same queue.
- `500 Internal Server Error`: a database operation failed; detailed database errors are logged on the server.

The API validates the shape of `kind` but does not maintain a supported-handler registry. The bundled worker implements only `echo` and `sha256`. An accepted unknown kind fails during processing and eventually exhausts its retry budget.

Demo payload controls

`demo_fail_until` makes the handler fail through the specified attempt number. `demo_sleep_ms` delays each attempt and must be between 0 and 60,000. These controls exist to make retries, heartbeat renewal, and crashes observable; they are not a general scheduling language.

The API wraps requests with a 10-second timeout. An accepted job runs asynchronously in workers, so that timeout does not limit the total job duration. An HTTP error or lost response can be ambiguous: reuse the original key when retrying.

12 / HTTP API: inspect and simulate

Method and route	Behavior
`GET /healthz`	Database ping: 200 `ok` or 503 `unavailable`. It does not verify that a worker is running.
`GET /v1/jobs`	Queue list; `queue=default`, `limit=100` by default. Limit range 1-1,000.
`GET /v1/jobs/{id}`	Job detail; 404 if absent.
`GET /v1/jobs/{id}/result`	Committed result; 404 if no result exists, including unfinished or unknown jobs.
`GET /v1/overview`	Whole-queue counts and a filtered page of jobs.
`POST /v1/simulations`	Runs an isolated bounded simulation and returns a report.
`GET /`	Embedded dashboard and its allowlisted static assets.

Overview query parameters

`queue` defaults to `default`. `status` is empty or one of the four job states. `search` is a literal, case-insensitive substring of ID, key, or kind, up to 256 bytes. `%` and `_` have no wildcard meaning. `offset` is 0-1,000,000; `limit` is 1-100 and defaults to 20.

The response includes `queue`, `stats`, `jobs`, `filtered`, `offset`, `limit`, and `server_time`. Counts in `stats` cover the whole queue, independent of the selected filter or page. Jobs are sorted newest first by creation time and ID. PostgreSQL uses a read-only repeatable-read transaction for a coherent response. Separate page requests can observe new inserts; there is no cross-request snapshot or cursor guarantee.

Run a simulation

```
curl -sS http://127.0.0.1:8080/v1/simulations \
  -H 'Content-Type: application/json' --data-binary @- <<'JSON'
{"seed":42,"steps":2000,"jobs":30,"workers":5}
JSON
```

Supply all four fields. HTTP limits are 100-10,000 steps, 1-200 jobs, 1-32 workers, and a JavaScript-safe integer seed. The request body limit is 4 KiB. One simulation is admitted at a time per server instance; overlapping requests receive 429. Invalid requests receive 400. The CLI has different, broader limits.

The simulation does not write to the live PostgreSQL queue. Its report includes accepted/succeeded/dead counts, claims, fault counters, stale rejections, and invariant checks.

Access boundary

This API has no authentication, tenant authorization, or TLS termination. Keep the supplied stack local. Its content-security policy and response headers protect browser behavior but do not provide access control.

13 / Dashboard walkthrough

Open `http://127.0.0.1:8080` after starting the stack. Go embeds `index.html`, `styles.css`, `app.js`, and the favicon; no separate frontend process is required. The console refreshes every two seconds.

Overview

The overview shows queue totals and counts for pending, running, succeeded, and dead jobs, plus a distribution and recent jobs. These totals refer to the selected queue, not just the visible page. Empty queues offer a button that submits six real demo jobs.

New job

Choose a queue, handler, idempotency key, retry budget, and delay. Enter valid JSON. The presets demonstrate a normal echo, two retries, a slow job, and failure. Keep the same key to demonstrate producer deduplication; generate a new key to create new work.

The form's JSON validation catches syntax errors before submission. The API still performs its own validation. The returned status may change quickly when a worker is already polling.

All jobs and job details

Search by ID, key, or handler; filter by state and browse pages. Open a job to inspect payload, attempts, lease owner/version, dates, last error, and committed result. A missing result is expected for jobs that have not succeeded.

Changing the queue field only changes which queue the GUI inspects and submits to. It does not start a worker for that queue. The bundled Compose worker consumes `default`; a custom queue requires a worker configured with the matching `-queue` flag.

Simulation lab

Choose a seed, event count, job count, and worker count. Run the lab and inspect the final accounting and injected-fault counters. Export the report as JSON for comparison or an interview demonstration. Use identical inputs with the same implementation to reproduce a run.

A five-minute presentation

1. Submit an echo job and inspect its saved result.
2. Repeat its key and payload to show the same job ID.
3. Change the payload under that key to show a conflict.
4. Use the retry preset and inspect the increasing attempt count.
5. Use the failure preset and show the retained dead job.
6. Run a seeded simulation and explain `accepted = succeeded + dead` after draining.

Browser validation previously exercised rendering, sample jobs, invalid JSON, submission, result inspection, simulation, and JSON export. Responsive CSS exists; dedicated mobile-viewport verification was not completed. See `docs/VALIDATION.md` for the recorded scope.

14 / Deterministic simulation

The simulator is a controlled model of queue behavior. It does not kill the live server. `simulation.Run` uses a seeded random generator, deterministic IDs, and an injected database clock in the memory store. It uses no real sleeps, goroutines, network calls, or system-clock reads.

```
go run ./cmd/dq simulate \
  -seed 42 -steps 10000 -jobs 100 -workers 8
go run ./cmd/dq simulate \
  -seed 42 -steps 1000 -jobs 30 -workers 5 -trace \
  > report.json 2> replay.log
```

Faults and actions

Event	What is modeled
Worker crash	The active in-memory assignment disappears; its lease remains.
Duplicate delivery	An old lease token is delivered again for completion.
Duplicate enqueue	A producer resubmits an existing logical request.
Lost acknowledgement	Completion is replayed with a changed proposed value.
Transient handler failure	The job follows retry or dead-letter rules.
Clock jump	Reference database time moves forward or backward.
Claim / heartbeat / completion	Ordinary operations interleave with the faults.

Assertions after every event

The simulator checks that accepted jobs do not disappear, unexpected jobs or results do not appear, each success has one result, committed results stay unchanged, attempts stay within budget, running status matches lease presence, versions and attempts do not decrease, and terminal states do not change. Its completion oracle rejects expired or superseded grants.

After faults stop, the simulator advances time and drains eligible work. Every accepted job must end in either `succeeded` or `dead`. Dead-lettering counts as accounted-for terminal failure, not successful work.

What reproducibility means

The same configuration and implementation produce the same report and trace. Preserve the seed, steps, jobs, workers, and source version when reporting a failure. A seed by itself does not describe a complete reproduction.

Simulation explores finite schedules against a reference implementation and shared transitions. It does not prove every possible execution or simulate PostgreSQL WAL, locking, disks, or network failures. Real-database tests and real-process smoke tests cover different parts of the system.

15 / Tests and recorded evidence

Run the checks

```
make test
make vet
export TEST_DATABASE_URL=\
'postgres://queue:queue@127.0.0.1:55432/queue?sslmode=disable'
make integration
go test -race -count=1 ./...
```

Without `TEST_DATABASE_URL`, PostgreSQL tests skip explicitly. Integration tests create and drop isolated schemas and need a development database with permission to do so. `make integration` selects the PostgreSQL package; the final command above runs the full suite with database tests enabled.

Layer	Main responsibility
Shared store contract	Deduplication, conflict handling, leases, retries, dead letters, concurrency
PostgreSQL tests	Atomic business-write rollback, replay, skipped row locks, concurrent consumers
Worker tests	Long-job renewal, panic handling, cooperative cancellation
HTTP/dashboard tests	Validation, routes, inspection, full counts, paging, result lookup, simulation isolation
Seeded simulation	200 seeds x 1,000 injected events, plus final drain checks
Process smoke	Actual worker SIGKILL recovery, increased version, one saved result

Additional reproducible checks

```
make build
# Load DATABASE_URL from .env before running the smoke script.
python3 scripts/smoke.py
go test ./simulation -run '^$' \
  -fuzz FuzzSchedules -fuzztime 30s
```

Existing verification record: 18 October 2026

The saved record reports passing race-detector tests with PostgreSQL, `go vet`, Docker startup, API demonstrations, actual worker-kill recovery, and database restart persistence. The GUI changes were followed by a full race-detector suite. See `docs/validation/gui-tests.txt` and `docs/VALIDATION.md`.

The saved 10,000-event seed-42 report accounts for 100 jobs: 69 succeeded and 31 dead. It includes 78 active-worker crashes, 1,019 clock jumps, 1,044 duplicate deliveries, and 847 stale rejections. These are simulator counters, not real production incidents or throughput figures.

The GitHub Actions workflow is supplied but has not been executed on GitHub in the recorded verification. The stored coverage profile predates the GUI. No production deployment, load benchmark, or formal correctness proof is claimed.

16 / Extending the project

Add a computation handler

Begin with `queue.Handler`, whose signature accepts a context and job and returns JSON plus an error. The current dispatch is `demoHandler` in `cmd/dq/main.go`. For a larger application, move business handlers into their own package and keep the CLI focused on startup.

```
// Illustrative handler shape; integrate it into your dispatch.
func handle(ctx context.Context, j queue.Job) (json.RawMessage, error) {
    if err := ctx.Err(); err != nil {
        return nil, err
    }
    // Validate input; perform context-aware computation.
    return json.Marshal(map[string]string{"status": "processed"})
}
```

Return an error for a retryable processing failure. The current protocol treats handler errors through the same bounded retry policy; there is no separate permanent-error type. Make long-running operations context-aware and design the computation to tolerate another attempt.

Add database business effects

Use the PostgreSQL `CompleteTx` integration path described on page 11. The application owns its business schema. Use only the supplied transaction for business writes, so those writes, the result, and completion commit atomically. Add a rollback test and a replay test proving the callback does not apply the change twice after a successful commit.

Do not call an external API inside that transaction. If adding email or payment integrations, define a destination idempotency key or an outbox/inbox protocol first and test the failure windows explicitly.

Change queue semantics carefully

1. Update shared types and transition rules in `queue/queue.go`.
2. Apply the same behavior to memory and PostgreSQL implementations.
3. Add a behavioral contract test that both stores must pass.
4. Extend simulator events or invariants if the protocol changed.
5. Update API validation and dashboard behavior where needed.
6. Run the race detector and real PostgreSQL tests; refresh documentation.

Schema changes require an explicit migration plan for existing databases. Adding SQL to `CREATE TABLE IF NOT EXISTS` will not alter an already-created table. Retention and dead-letter replay also need clear rules for key lifetime, attempt budgets, and result history.

Frontend edits live under `internal/httpapi/web/`. Rebuild the binary or Docker image to include changed embedded files. Optional JavaScript syntax check: `node --check internal/httpapi/web/app.js`.

17 / Operations and troubleshooting

Observe before changing state

```
docker compose ps
docker compose logs --tail=100 api worker postgres
curl http://127.0.0.1:8080/healthz
```

Symptom	Likely cause and next check
Connection refused on port 8080	API is stopped, build failed, or startup migration failed. Inspect Compose status and logs.
API healthy, jobs remain pending	Health checks the database only. Check worker status, queue name, and future `available_at`.
Running job appears stuck	Check lease renewal and worker logs. Recovery needs a polling worker and an expired lease.
Job becomes dead	Inspect `last_error`, handler kind, and attempts. A crash also consumes an attempt.
HTTP 409	The same queue/key was reused for a different kind or payload. Use the original payload or a new logical-request key.
Result endpoint returns 404	The job has no committed result yet, is dead, or the ID is unknown. Inspect job status first.
Custom queue is idle	Start a worker with the same `-queue`; changing the GUI queue does not configure workers.
Tests pass suspiciously quickly	Check output for skipped PostgreSQL tests and set `TEST_DATABASE_URL`.
Dashboard still shows old code	Rebuild the embedded assets with `make build` or `docker compose up --build -d`.

Restart and crash demonstrations

An ordinary `docker compose down` retains the named data volume. Worker recovery requires a new worker process, not just restarting the API. For a controlled crash demo, use `docs/DEMO.md` or `scripts/smoke.py`; the smoke script uses a unique queue and leaves its results available for inspection.

Current operational limits

There is no automatic cleanup, dead-letter replay endpoint, priority scheduler, tenant authorization, guaranteed global ordering, or metrics backend. Expired final attempts are swept in batches of 100 during claims, so an idle queue without workers does not sweep itself.

Database durability depends on PostgreSQL WAL and storage. The project does not implement backups, replication, failover, or disaster recovery. If both the process and its only durable data are destroyed, lease logic cannot recreate the lost data.

Before exposing a deployed service, design authentication, TLS, credential management, process supervision, backups, observability, and load testing for that environment. Publishing source code on GitHub is separate from exposing this development service to the internet.

18 / GitHub, LinkedIn, and interviews

Can this be a portfolio project?

Yes. It contains runnable code, a database-backed implementation, a working GUI, tests, a simulator, and documentation. Present it as a personal systems/backend project. Be prepared to run it and explain the failure cases and limits yourself.

Prepare the repository

- Use `README.md` as the landing page and link this PDF and its Markdown source.
- Include source, `go.mod`, `go.sum`, Compose, tests, the workflow, and documentation.
- Keep actual `.env` files, access tokens, private data, binaries, and scratch directories out of the repository. `.env.example` documents local development values.
- Choose an appropriate license if you intend to grant reuse permissions; no license has been selected by this project.
- Run the documented checks and inspect the GitHub Actions result after pushing. The existing local verification record is not a hosted CI badge.
- Add a screenshot or short demo recording from your own running instance. Do not imply a public deployment if only the repository is public.

Creating or publishing a remote repository is a separate action. This documentation does not create a GitHub repository or post to LinkedIn.

Suggested LinkedIn project description

> Built a background job queue using Go and PostgreSQL with expiring leases, retries, idempotency keys, and transactional result commits. Added a web dashboard for submitting and inspecting jobs, plus a deterministic simulator for replaying crashes, duplicate deliveries, and clock jumps. Validated behavior through shared contracts, PostgreSQL integration tests, and worker crash-recovery checks.

Project title: Durable Job Queue with Simulation Testing. Suggested skills: Go, PostgreSQL, SQL, concurrency, REST APIs, Docker, testing, and JavaScript. Add the actual repository URL when it exists.

Explain the motivation simply

> Applications often need to finish work after the user leaves the page. I built this project to understand how that work can stay recorded and recover when a worker crashes, and how to test those failure cases reproducibly.

Answer the hard question accurately

If asked whether a job can run twice: yes, processing attempts can repeat; the queue prevents duplicate committed database results. Leases allow recovery, fencing rejects superseded workers, and a transaction makes result storage and completion atomic. External effects need additional design.

Avoid unsupported claims about production traffic, throughput, formal proof, or universal exactly-once execution. Describe the work and tools you used honestly, and demonstrate your understanding through the code and tests.

19 / Glossary and maintenance

Term	Plain meaning
Job / producer / worker	Saved work / the caller submitting it / the process handling it.
Durable	Stored beyond one process's memory, subject to database storage guarantees.
Lease	Temporary permission to handle a job, with a deadline.
Heartbeat	A renewal showing that the current worker is still active.
Fencing token	A version that distinguishes the current assignment from older ones.
Idempotency key	A stable key for safely resubmitting one logical request.
Transaction	A group of database changes committed together or rolled back together.
Backoff and jitter	A retry delay that grows, plus a small offset to spread retries.
Dead letter	A retained job that used all its allowed attempts.
Invariant	A condition that must remain true after every modeled event.
Deterministic replay	Reproducing the same schedule from the same configuration and code.

Reading the implementation

Start with `queue/queue.go`, then `queue/worker.go`, then the PostgreSQL store. Read `internal/queuetest/contract.go` to see intended behavior expressed as tests. Next inspect `simulation/simulation.go`, the HTTP routes, and the frontend. The repository files and saved evidence are the primary sources for this handbook.

Rebuild this document

The editable source is `docs/DEVELOPER_GUIDE.md`. The renderer is `scripts/build_docs.py`; its only third-party runtime dependency is ReportLab. Keep document dependencies separate from the Go application.

```
python3 -m venv .venv-docs
. .venv-docs/bin/activate
python -m pip install -r docs/requirements.txt
python scripts/build_docs.py
```

The output is `docs/DEVELOPER_GUIDE.pdf`. Page markers keep topics together. The renderer rejects content that unexpectedly spills onto another page; adjust the content or deliberate page plan when adding material. Render the PDF to images and inspect every page before publishing a changed edition.

Documentation update checklist

When behavior changes, update examples, limits, guarantees, API fields, and test evidence together. Re-run applicable checks before replacing recorded results. Keep the implementation date and verification scope explicit. Update this PDF, the README link, and any distribution archive after rebuilding.

This edition documents the project as inspected on 18 October 2026. It is a developer handoff and portfolio reference, not a claim of production deployment.